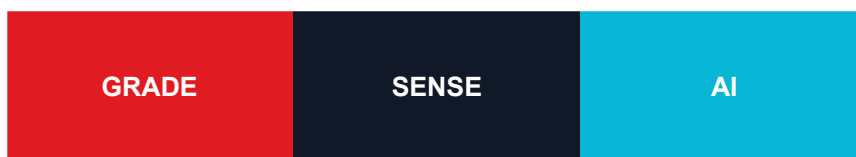


GradeSense Source Code Documentation

Curated architecture and syntax-highlighted algorithm documentation



Explainable predictions, constraint-aware recommendations, and real-time operations for safer, faster paper grade transitions.

Prepared from the validated GradeSense v0.2.0 source repository and live application.

Honeywell GradeSenseAI GradeSenseAI is an industrial decision-support platform for paper-grade transitions. It combines live process monitoring, multi-horizon Basis Weight forecasting, constraint-safe intervention simulation, operator workflow tracking, model governance, and production observability in one deployable application. This repository is the final Honeywell Hackathon submission candidate. Production demos load the supplied pre-trained artifacts and never train or generate data automatically during startup. The explicit /dataset/regenerate compatibility endpoint and offline scripts remain available for controlled development and experimentation. Honeywell challenge mapping | Manufacturing need | GradeSense capability | | --- | --- | | Detect transition risk before paper becomes off-spec | Direct multi-horizon Basis Weight forecast with $\pm 2.5\%$ specification monitoring | | Estimate when a deviation will occur | First-crossing step/time and remaining safe operating time |

This document summarizes the important implementation contracts and selected algorithmic patterns. It intentionally avoids a raw source dump.

Backend Structure

- app/api/routes: system, intelligence, real-time, forecasting, interventions, and administration.
- app/services: behavior for model inference, explainability, recommendations, forecasting, streaming, registry, operations, and constraints.
- app/schemas: versioned public request and response contracts.
- app/models and app/database: SQLAlchemy entities, sessions, and shared mixins.
- alembic: ordered database migrations through operational governance.

Service boundary pattern

```
route request -> validate schema
resolve dependency -> domain service
service -> active model + persistence boundary
return typed response with request ID
```

Frontend Structure

- app/App.tsx: lazy route composition inside the shared application shell.
- pages/: workflow screens for operations, intelligence, history, and platform administration.
- components/: reusable live monitoring, process forms, prediction result, layout, forecast, and UI states.
- api/: typed client and shared contracts; hooks/: WebSocket subscription.
- stores/: focused Zustand stores for process inputs and theme.

Client data flow

```
Route -> page component
page -> TanStack query or mutation
typed API client -> REST endpoint
useLiveStream -> WebSocket event reducer
result -> charts, status, explanation, action
```

Major Modules

GradeSense | Honeywell Hackathon Submission

- IntelligenceService: readiness, prediction, recommendation, persistence.
- ForecastingService: history window, feature construction, direct-horizon trajectory.
- InterventionEngine: candidate generation, constraint checks, simulation, ranking, lifecycle.
- StreamingService: single-writer replay, bounded queues, typed broadcast, rolling metrics.
- ModelRegistryService: immutable registration, checksum validation, promotion, archive.
- Operations services: audit, runtime configuration, health, metrics, and export.

Important Algorithms

GradeSense | Honeywell Hackathon Submission

Snapshot inference

```
validate(process_state)
features = preprocessor.transform(process_state)
model = registry.resolve_active(kind="snapshot")
prediction = model.predict(features)
explanation = explainability.describe(model, features)
persist(inputs, prediction, explanation, model.version)
```

Specification crossing

```
target_band = target_basis_weight * 0.025
for point in forecast_trajectory:
    deviation = abs(point.basis_weight - target_basis_weight)
    if deviation <= target_band:
        return first_crossing_time
    return not_crossed_within_horizon
```

ML Pipeline

GradeSense | Honeywell Hackathon Submission

- Snapshot features represent the current grade transition state.
- Sequential features add lags, rolling statistics, rates of change, grade pair, and transition stage.
- Direct horizon regressors predict each future point without autoregressive feedback.
- A crossing classifier and validation residual intervals quantify operational risk.
- Transition-ID boundaries reset history so training and inference windows remain valid.

Sequential feature window

```
assert samples are ordered
assert one transition_id per history window
lags = values[t-k : t]
rolling = mean/std/min/max(lags)
rates = diff(lags) / delta_time
features = concat(lags, rolling, rates, grade_pair, stage)
```

Recommendation Engine

GradeSense | Honeywell Hackathon Submission

- Generate meaningful set-point deltas for controllable variables.
- Reject out-of-range and incompatible combinations before model execution.
- Simulate each safe candidate using the active forecast artifact.
- Require a measurable improvement over the persisted baseline.
- Rank deterministically and retain the rationale, constraints, model version, decision, and outcome.

Constraint-aware ranking

```
for candidate in generate_candidates(baseline):
    if not constraints.valid(candidate): continue
    simulated = forecast.simulate(candidate)
    benefit = compare(simulated, baseline)
    if benefit.positive: accepted.append((candidate, benefit))
    return sorted(accepted, key=deterministic_priority)
```

Streaming Engine

GradeSense | Honeywell Hackathon Submission

- A single background worker reads the next ordered sample at a configurable cadence.
- Prediction, recommendation, alert, drift, and basis-forecast events share one envelope.
- Client queues and rolling buffers are bounded.
- Transition changes clear forecast history to prevent invalid cross-transition windows.
- Rolling metrics are periodically persisted for operations and audit.

Streaming loop

```
while running:
    sample = source.next()
    events = intelligence.evaluate(sample)
    await websocket.broadcast(events)
    rolling.update(sample, events)
    if rolling.snapshot_due: persist(rolling.snapshot())
```

Testing and Maintainability

GradeSense | Honeywell Hackathon Submission

- Backend pytest suites cover system, intelligence, real-time, forecasting, recommendations, interventions, datasets, and production readiness.
- Frontend Vitest suites cover live monitoring and end-to-end workflow states.
- Typed schemas and API client types reduce contract drift.
- Services isolate behavior from HTTP details, making algorithm tests deterministic.
- Immutable artifacts and audit persistence make operational behavior traceable.